



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Trabajo Final

Semestre 2026-2

Ingeniería en Computación

Cómputo Móvil

Grupo 03

Ing. Marduk Pérez de Lara Domínguez

Carreón Blas Andrea **318207560**

Hernández Cortés Armando Daniel **318241254**



Ciudad Universitaria, CDMX 29 Mayo, 2026

Índice

1. Introducción	4
2. Detalle de requerimientos de la aplicación.....	4
2.1 Reglas de Negocio.....	4
2.2 Requerimientos Funcionales.....	5
2.3 Requerimientos No Funcionales.....	5
3. Justificación de viabilidad e inviabilidad técnica.....	5
3.1 Funcionalidad Propuesta: Síntesis de Planes de Estudio Personalizados por IA en Tiempo Real.....	5
4. Alcance y producto mínimo viable (MVP).....	6
5. Wireframes de la aplicación.....	7
6. Explicación del flujo y recorrido de pantallas.....	7
7. Documentación técnica detallada por pantalla.....	9
Pantalla P01: Pantalla de carga inicial.....	9
Pantalla P02: Introducción guiada.....	9
Pantalla P03: Portal de Autenticación.....	9
Pantalla P04: Panel principal de control.....	10
Pantalla P05: Cuestionario de Contexto Inicial.....	10
Pantalla P06: Chatbot de Orientación Vocacional.....	11
Pantalla P07: Pantalla de Procesamiento IA.....	11
Pantalla P08: Mapa de ruta y resultados.....	12
Pantalla P09: Detalle Curricular y Laboral.....	12
Pantalla P10: Panel de Usuario y Ajustes.....	13
8. Análisis de flujo y cambios de estado.....	13
9. Análisis de flujo y cambios de estado.....	14
10. Dispositivos objetivo, tamaños y orientaciones.....	15
11. Maqueta final y vistas de alta fidelidad.....	15
12. Maqueta final y vistas de alta fidelidad.....	16
13. Arquitectura, metodologías y patrones de diseño.....	17
13.1 Patrón Arquitectónico Base: MVVM (Model-View-ViewModel).....	18
13.2 Patrones de Diseño Implementados.....	18
14. Administración de proyectos.....	18
15. Uso de inteligencia artificial en el desarrollo.....	19
16. Frameworks, kits y herramientas nativas.....	19
17. Permisos y consideraciones para la app store Frameworks, kits y herramientas nativas.....	20
17. Equipo de trabajo y roles.....	20
18. Estimaciones de tiempo de desarrollo y costos.....	21

18.1 Desglose de Esfuerzo Temporal.....	21
18.2 Proyección Financiera Operativa.....	22
19. Referencias bibliográficas	23

1.Introducción

El presente documento detalla la elaboración y el diseño de la aplicación móvil studAI. La idea central se basa en la crisis estructural en el sistema educativo mexicano; de acuerdo con cifras oficiales de la Universidad Nacional Autónoma de México, 7 de cada 10 estudiantes a nivel bachillerato no saben que carrera universitaria estudiar, y una vez seleccionada, entre el 30% y el 40% de los jóvenes universitarios se equivocan en su elección. La solución que se propone con la aplicación radica en redefinir la orientación vocacional, alejándonos de los cuestionarios estáticos tradicionales mediante la implementación de Inteligencia Artificial Generativa y un desarrollo nativo optimizado para el ecosistema de Apple.

El nombre de la aplicación está compuesto por la fusión entre las palabras en inglés *student* (estudiante) y el acrónimo *AI* (Inteligencia Artificial). Apoyándose en el paradigma de Inteligencia Artificial Centrada en el Humano, la plataforma actúa como un facilitador del autodescubrimiento donde la tecnología colabora y empodera a las personas sin reemplazarlas. El usuario siempre tiene el control y guía la conversación de forma libre; si la IA le sugiere ingeniería en computación pero el alumno responde que detesta estar encerrado programando, el sistema recalcula y ofrece alternativas acordes.

2. Detalle de requerimientos de la aplicación

2.1 Reglas de Negocio

Establecen los límites operativos y éticos del sistema, garantizando la integridad de los datos del usuario y la plataforma.

- **Privacidad y autonomía del estudiante:** Toda la información recopilada en los diálogos del chatbot y cuestionarios se procesa de forma estrictamente anónima. Queda prohibida la comercialización de perfiles y el uso de las conversaciones para entrenar modelos de lenguaje público.
- **Transparencia y control del estudiante:** El sistema utiliza el principio de IA Centrada en el Humano. El usuario lleva el control en todo momento, la IA no emite veredictos definitivos, está obligado a argumentar el razonamiento detrás de cada sugerencia. Los alumnos pueden considerar o descartar las sugerencias en cualquier momento.
- **Límite de cuotas:** El acceso al sistema es ilimitado para el alumno, está condicionado a que su correo pertenezca al dominio institucional de su escuela y pagado por parte de la universidad. Queda prohibida cualquier pago
- **Catálogo de carreras:** Las carreras sugeridas por la IA deben de pertenecer estrictamente a los planes de estudio reales, vigente y que cuenten con reconocimiento

de validez o pertenezcan a las retículas de las universidades a las que se asocien.

2.2 Requerimientos Funcionales

- **Gestión de Identidad Institucional:** El sistema permitirá el acceso mediante el Single Sign-On (SSO) de la universidad y, obligatoriamente por normativas de Apple, el botón nativo **Sign in with Apple**.
- **Cuestionario de Contexto Inicial:** Registro rápido donde el estudiante indica su bachillerato de procedencia (ej. Prepa o CCH) y fecha de nacimiento para activar las reglas de minoría de edad.
- **Chatbot:** Interfaz conversacional fluida guiada por IA que reemplaza los test estáticos para mapear habilidades y gustos.
- **Mapa de Ruta y Resultados Vocacionales:** Mapeo interactivo visual que le indica al alumno los pasos y planes de estudio oficiales de su institución para iniciar la carrera.
- **Fichas del Mercado Laboral:** Visualización en el Perfil de los salarios reales en México, demanda laboral futura y materias principales.
- **Módulo de Eliminación de Cuenta:** Botón nativo en el perfil que permite al alumno borrar físicamente su cuenta, historial de chats con la IA y tokens de Apple de los servidores..

2.3 Requerimientos No Funcionales

Establecen los criterios de calidad, rendimiento y seguridad de la arquitectura de software:

- **Latencia de Inferencia:** El tiempo total desde que el usuario solicita la generación de su *Roadmap* hasta que el backend devuelve y procesa la respuesta del LLM no debe exceder los 4.0 segundos en conexiones 4G/LTE.
- **Rendimiento Gráfico Nativo:** La interfaz desarrollada en SwiftUI mantendrá una tasa estable de 60 cuadros por segundo (FPS), garantizando transiciones suaves y soporte nativo optimizado para Modo Oscuro.
- **Seguridad en Tránsito y Reposo:** Todas las peticiones HTTP/REST estarán cifradas bajo el protocolo TLS 1.3. Los JSON Web Tokens (JWT) de sesión se almacenarán encriptados en el *Keychain* nativo de iOS.
- **Disponibilidad y Concurrencia:** El middleware desarrollado en Flask soportará peticiones asíncronas concurrentes, asegurando un tiempo de actividad (*uptime*) del 99.0%.

3. Justificación de viabilidad e inviabilidad técnica

3.1 Funcionalidad Propuesta: Síntesis de Planes de Estudio

Personalizados por IA en Tiempo Real

Durante las fases de ideación del Hackathon, se propuso que la IA no solo recomendara carreras existentes, sino que combinara dinámicamente retículas de múltiples licenciaturas para "inventar" una carrera sintética hecha a la medida de los gustos específicos de cada alumno.

Veredicto Técnico: Estrictamente Inviabile para su liberación comercial.

Justificación Exhaustiva de Inviabilidad:

1. **Incompatibilidad de Dominio:** El objetivo real de un estudiante de preparatoria es matricularse en una institución oficial tras superar un examen de admisión. Generar un plan curricular inventado carece de validez oficial (RVOE) y genera frustración al no existir una oferta laboral formal ni una facultad real asociada.
2. **Alucinaciones del Modelo:** Los LLMs tienen un margen de alucinación donde podrían inventar secuencias lógicas de materias sin sentido pedagógico o dependencias curriculares imposibles de validar.
3. **Costo de Latencia y Tokens de Salida:** Redactar un plan de estudios completo de más de 40 materias con descripciones semestrales requiere miles de tokens de salida (*output tokens*). Esto elevaría drásticamente la facturación de la API a niveles insostenibles para una app gratuita y dispararía el tiempo de respuesta por encima de los 15 segundos, arruinando la experiencia de usuario.

Alternativa Técnica Implementada (Viable): Se implementó un modelo de **Clasificación y Filtrado Contextual**. La IA actúa como un motor de inferencia semántica que evalúa el perfil conversacional del alumno y devuelve únicamente identificadores clave de compatibilidad en formato JSON. El backend utiliza estos metadatos para consultar una base de datos propia pre-poblada con información verídica, oficial y estructurada del catálogo de carreras de la UNAM y el IPN.

4. Alcance y producto mínimo viable (MVP)

El alcance de **studAI** se enfoca directamente en atender a estudiantes mexicanos que busquen una orientación vocacional, un segmento de nativos digitales que valora la inmediatez, las interfaces limpias y rechaza los formatos largos e institucionales.

Dentro del alcance del MVP:

- **Inicio de sesión y registro unificado:** Acceso rápido y directo.
- **Incorporación contextual:** Pantallas iniciales rápidas para capturar el semestre y área de estudio actual del usuario.
- **Chatbot:** Entorno conversacional donde el estudiante interactúa con la IA
- **Pantalla de resultados dinámicos:** Visualización inmediata del Top 3 de carreras

compatibles usando gráficas de radar.

- **Fichas técnicas:** Datos concretos de cada carrera sugerida, mostrando salarios reales en México y planes de estudio básicos pre-cargados.

Fuera del alcance del MVP:

- Conexión directa con los portales de inscripción al examen de selección de la UNAM (DGAE).
- Redes sociales o foros comunitarios internos entre aspirantes.

5. Wireframes de la aplicación

El diseño esquemático contempla la siguiente estructura visual para cubrir de extremo a extremo la experiencia del aspirante:

1. **Pantalla inicial:** Interfaz de arranque. Muestra el logotipo de la aplicación studIA.
2. **Introducción Guiada Informativa:** Carrusel visual de 3 páginas explicando cómo la IA actúa como copiloto vocacional. Incluye un botón para omitir la introducción.
3. **Portal de autenticación:** Interfaz de acceso rápido. Contiene los campos estándar para correo electrónico y contraseña.
4. **Dashboard Principal (Home):** Tarjeta de bienvenida superior, botón de acción primario "Hablar con mi Mentor IA" y carrusel inferior de carreras destacadas.
5. **Cuestionario de Contexto Inicial:** Vista de captura rápida para registrar el semestre en curso y el área de bachillerato del alumno.
6. **Chatbot:** Interfaz conversacional limpia con burbujas de mensaje diferenciadas y barra inferior de entrada de texto.
7. **Pantalla de Procesamiento IA:** Vista de transición visual. Su objetivo es mitigar el tiempo de espera mostrando una animación de carga fluida acompañada de datos curiosos sobre el mercado laboral mexicano.
8. **Mapa de Ruta y Resultados Vocacionales:** Despliegue visual superior con gráfica poligonal nativa y tarjetas colapsables con el Top 3 de carreras compatibles.
9. **Detalle Curricular y Laboral:** Pestañas de navegación interna mostrando Información General, Retícula de Materias y Salarios Promedio.
10. **Panel de Usuario y Ajustes:** Gestión del perfil, consulta del historial local de *Roadmaps* generados, términos de privacidad y cierre de sesión.

6. Explicación del flujo y recorrido de pantallas

La navegación de la aplicación sigue una estructura jerárquica combinada con flujos modales de alta retención para asegurar que el usuario complete su ciclo de descubrimiento sin distracciones.



Mecánica de Recorrido:

El usuario arranca la vista entrando a la aplicación **P01**, la cual validará la sesión en segundo plano. Si el usuario es nuevo le dará una introducción **P02** y posteriormente mostrará el portal de autenticación **P03**. Una vez en la pantalla de Control **P04**, se aplicará un cuestionario inicial para conocer al alumno **P05** dando inicio al chat en vivo **P06** para que así el estudiante tenga una conversación y decidir generar el roadmap. Esto detona la vista de procesamiento (**P07**), bloqueando la interfaz para evitar peticiones duplicadas. Al recibir la respuesta del backend, se inyectan los datos en la vista de Resultados (**P08**). Desde allí, el alumno puede profundizar en las materias (**P09**) o regresar al Home para consultar su historial local.

7. Documentación técnica detallada por pantalla

En el siguiente apartado se detalla la matriz de ingeniería para cada una de las pantallas, abarcando su capa de datos, almacenamiento y servicios en la nube.

Pantalla P01: Pantalla de carga inicial

- **Funcionalidad:** Inicialización del motor de la app, verificación de sesión activa y carga de recursos gráficos estáticos.
- **Información y Esquema de Datos:** No captura datos del usuario. Evalúa la bandera booleana interna `isLoggedIn`.
- **Vigencia y Origen:** Inmediata al arranque (1.5 segundos). Origen local.
- **Operaciones Esperadas:** Lectura de variables de entorno locales.
- **Servicios Cloud / APIs:** Ninguno.
- **Almacenamiento Local:** Lectura de `UserDefaults` y extracción segura del token de sesión desde el **Keychain** de iOS.

Pantalla P02: Introducción guiada

- **Funcionalidad:** Encaminar al alumno sobre las capacidades de la IA como copiloto vocacional para orientar a los alumnos.
- **Información y Esquema de Datos:** Textos estáticos descriptivos e ilustraciones vectoriales. Variable de estado `currentPage` (Entero).
- **Vigencia y Origen:** Estática, embebida de forma nativa en el código compilado.
- **Operaciones Esperadas:** Desplazamiento de vistas mediante gestos horizontales.
- **Servicios Cloud / APIs:** Ninguno.
- **Almacenamiento Local:** Al finalizar el flujo, se ejecuta una operación de escritura local en `UserDefaults` asignando `hasCompletedTutorial = true` para omitir esta pantalla en el futuro.

Pantalla P03: Portal de Autenticación

- **Funcionalidad:** Identificar y autorizar unívocamente al alumno dentro de la plataforma.
- **Información y Esquema de Datos:**
 - email: String (Validación local por Regex).
 - password: String (Oculto en campo seguro).
 - ssoToken: String (Generado por SDKs de Apple o Google).
- **Vigencia y Origen:** Captura manual o inyección por SDK. Vigencia en memoria hasta completar la petición HTTP.
- **Operaciones Esperadas:** Envío de credenciales (POST), recepción de perfil básico y

token de acceso.

- **Servicios Cloud / APIs:**
 - **Tipo de Servicio:** Operación de Consulta y Registro.
 - **Endpoint:** POST /api/v1/auth/login
 - **Características:** Implementado en el middleware de Flask. Recibe JSON y retorna un JSON Web Token (JWT) firmado con vigencia de 30 días.
- **Almacenamiento Local:** El token JWT devuelto se almacena de forma persistente y altamente cifrada en el **Keychain** del sistema operativo nativo, impidiendo su extracción en texto plano.

Pantalla P04: Panel principal de control

- **Funcionalidad:** Centro de control del aspirante. Despliega un saludo personalizado, accesos rápidos al test conversacional y recomendaciones generales.
- **Información y Esquema de Datos:**
 - **userName:** String.
 - **remainingCredits:** Integer.
 - **popularCareers:** Array de objetos CareerSummary (ID, Nombre, Imagen, Facultad).
- **Vigencia y Origen:** Origen remoto en base de datos. Vigencia por refresco de vista o acción de arrastre (*pull-to-refresh*).
- **Operaciones Esperadas:** Lectura de datos de perfil y catálogos.
- **Servicios Cloud / APIs:**
 - **Tipo de Servicio:** Consulta REST.
 - **Endpoint:** GET /api/v1/student/dashboard
 - **Autenticación:** Requiere inyectar en la cabecera HTTP el parámetro Authorization: Bearer <JWT>.
- **Almacenamiento Local:** Se almacena un caché estructural del último estado del dashboard utilizando **SwiftData** para permitir que la vista principal cargue de forma instantánea al abrir la app en modo offline parcial.

Pantalla P05: Cuestionario de Contexto Inicial

- **Funcionalidad:** Capturar parámetros clave sobre la situación académica actual del alumno para proveer un punto de partida lógico al motor de IA.
- **Información y Esquema de Datos:**
 - **currentSemester:** Integer (1 al 6).
 - **highSchoolTrack:** String (Físico-Matemáticas, Médico-Biológicas, etc.).
 - **mainInterests:** Array de Strings.
- **Vigencia y Origen:** Selección manual en pantalla. Vigencia temporal vinculada a la sesión de preparación del chat.

- **Operaciones Esperadas:** Almacenamiento en memoria volátil del estado global.
- **Servicios Cloud / APIs:** Ninguno directo; los datos se empaquetan en un payload para su envío posterior.
- **Almacenamiento Local:** Se retiene temporalmente en la memoria RAM del ViewModel reactivo.

Pantalla P06: Chatbot de Orientación Vocacional

- **Funcionalidad:** Interfaz conversacional bidireccional donde el usuario dialoga de forma natural con la IA para explorar sus pasiones e inquietudes laborales.
- **Información y Esquema de Datos:**
 - chatMessages: Array de objetos MessageBubble conteniendo id (UUID), role (Enum: user o model), text (String) y timestamp (Date).
 - inputText: String.
- **Vigencia y Origen:** Generación continua por el usuario y el servidor web. Vigencia persistente por sesión de chat.
- **Operaciones Esperadas:** Inserción en tiempo real de nuevos nodos de mensaje y renderizado dinámico en lista.
- **Servicios Cloud / APIs:**
 - **Tipo de Servicio:** Interacción continua de Registro y Consulta.
 - **Endpoint:** POST /api/v1/chat/turn
 - **Características:** Utiliza JSON sobre REST o flujos continuos (*Server-Sent Events*) para animar la escritura del bot en tiempo real.
- **Almacenamiento Local:** Cada mensaje enviado o recibido se guarda asíncronamente en **SwiftData** para no perder el hilo conversacional si la aplicación se cierra inesperadamente.

Pantalla P07: Pantalla de Procesamiento IA

- **Funcionalidad:** Retener la atención del estudiante y gestionar la latencia de red mientras el backend realiza la llamada de inferencia al LLM.
- **Información y Esquema de Datos:** Envío del payload completo combinando el contexto inicial y el historial de chat compilado.
- **Vigencia y Origen:** Transitoria (3.0 a 5.0 segundos).
- **Servicios Cloud / APIs (Conexión Integral con Google Gemini API):**
 - **Características del Servicio:** El cliente móvil realiza un POST al endpoint /api/v1/analysis/generate. El servidor Flask intercepta la petición, inyecta el *System Prompt* maestro y se comunica de forma segura con la API de **Google Gemini 1.5 Flash**.
 - **Autenticación:** API Key administrada de forma totalmente segura en las variables

de entorno del servidor backend (nunca expuesta en el cliente iOS).

- **Formatos:** Se fuerza una salida estructurada inyectando el parámetro `response_mime_type: "application/json"` en la petición a Gemini para garantizar un retorno predecible sin texto adicional.
- **Costos de Conexión:** El consumo comercial de la API de Gemini factura a razón de **\$0.075 USD** por millón de tokens de entrada y **\$0.30 USD** por millón de tokens de salida. El consumo promedio por análisis es de 1,200 tokens totales.
- **Almacenamiento Local:** Ninguno.

Pantalla P08: Mapa de ruta y resultados

- **Funcionalidad:** Renderizar visualmente el dictamen JSON devuelto por la IA, mostrando gráficas de compatibilidad y el Top 3 de carreras afines.
- **Información y Esquema de Datos:**
 - `roadmapId`: UUID.
 - `radarMetrics`: Diccionario de Float (ej. `["Tecnología": 0.92, "Sociales": 0.40]`).
 - `recommendedCareers`: Array de objetos con ID oficial, Nombre, Porcentaje de Match y Justificación corta.
- **Vigencia y Origen:** Obtenido directamente de la respuesta de P07. Vigencia permanente en el historial.
- **Operaciones Esperadas:** Trazado de gráficos vectoriales nativos. Registro de persistencia final.
- **Servicios Cloud / APIs:** Los datos se inyectan desde la respuesta resuelta en el paso anterior sin requerir nuevas peticiones de red.
- **Almacenamiento Local:** El JSON exitoso se mapea a entidades de Swift y **se persiste de forma definitiva en SwiftData** para permitir que el estudiante revise sus resultados offline en el futuro.

Pantalla P09: Detalle Curricular y Laboral

- **Funcionalidad:** Desglosar información dura y oficial sobre la carrera seleccionada para contrastar la sugerencia de la IA con la realidad del mercado.
- **Información y Esquema de Datos:**
 - `careerName`: String.
 - `description`: String.
 - `entrySalaryMXN`: Double.
 - `syllabus`: Array de asignaturas agrupadas por semestre.
- **Vigencia y Origen:** Base de datos estática del backend.
- **Operaciones Esperadas:** Lectura y guardado opcional en lista de seguimiento.
- **Servicios Cloud / APIs:**

- **Tipo de Servicio:** Consulta estática (GET).
- **Endpoint:** GET /api/v1/careers/details/{id} (Servido a través de caché CDN).
- **Almacenamiento Local:** Se emplea el sistema nativo de almacenamiento en caché para imágenes y logotipos de facultades a través del sistema de archivos local para optimizar datos móviles.

Pantalla P10: Panel de Usuario y Ajustes

- **Funcionalidad:** Administrar las credenciales de la cuenta y consultar el listado de diagnósticos pasados.
- **Información y Esquema de Datos:** Preferencias de interfaz (Modo Oscuro/Claro), datos de perfil y referencias a la base local.
- **Operaciones Esperadas:** Actualización de datos (PUT), borrado de historial (DELETE) y cierre de sesión.
- **Servicios Cloud / APIs:**
 - **Endpoints:** PUT /api/v1/student/update y DELETE /api/v1/student/account (Servicio de eliminación obligatoria de cuenta exigido por normativas de tiendas de apps).
- **Almacenamiento Local:** Al ejecutar el cierre de sesión, se invoca un borrado estricto que destruye el token en el *Keychain* y purga la base de datos local *SwiftData*.

8. Análisis de flujo y cambios de estado

El flujo de la aplicación se centra en un patrón de datos unidireccional, implementando una arquitectura altamente optimizada para aplicaciones nativas de Apple que integran Inteligencia Artificial Generativa. Se hace uso de las capacidades del framework nativo y la macro `@Observable` de Swift. El ciclo de vida del flujo de datos se divide de manera rigurosa en cuatro etapas fundamentales:

1. Datos del modelo (Centralización del Estado Inicial) El estado inicial y global de la aplicación (como el historial del chatbot, las selecciones del usuario, el texto del *prompt* y las banderas de carga) se define y centraliza en un **ViewModel**. Este diseño establece una única fuente de la verdad, asegurando que los datos fluyan hacia las vistas de manera estrictamente unidireccional y evitando discrepancias de información entre pantallas.

2. Interacción del usuario (Captura y Emisión de Eventos) Las vistas de SwiftUI actúan como componentes pasivos. Cuando el estudiante interactúa con la plataforma, la vista no modifica directamente el estado interno. En su lugar, la interfaz captura esta intención y emite un evento hacia el **ViewModel**, delegando toda la responsabilidad algorítmica a la capa lógica.

3. Generación con IA (Inferencia y Procesamiento Asíncrono) Una vez que el **ViewModel** recibe y procesa el evento del usuario, orquesta la llamada a la API de IA generativa. Durante este proceso, el estado muta para indicar que el sistema está trabajando. Cuando la IA termina de evaluar el perfil y devuelve un resultado, el **ViewModel** decodifica la información y actualiza el estado general de la aplicación.

4. UI Reactiva (Re-renderizado Automático y Declarativo) La principal ventaja de este ecosistema es su sistema de observación. SwiftUI "escucha" y observa continuamente el estado del **ViewModel**. En el instante en que el estado es actualizado con la respuesta de la IA, el framework re-renderiza automáticamente la vista con el nuevo contenido generado. En studAI no hay manipulación manual de la UI todo el trazado visual de los resultados y mensajes del chat depende de forma absoluta y reactiva de los datos del estado.

Ciclo de Vida de los Estados de Interfaz:

- **State.idle:** La vista se encuentra en reposo absoluto, reflejando el estado inicial del modelo a la espera de una interacción manual por parte del estudiante.
- **State.loading:** Durante la etapa de "Generación con IA", el estado bloquea los controles táctiles para evitar envíos duplicados y despliega indicadores visuales nativos (ProgressView).
- **State.success(Data):** La respuesta de la IA es procesada por el ViewModel. La UI reactiva detecta el cambio e inyecta inmediatamente el contenido generado en la pantalla.
- **State.error(Message):** Ante cualquier fallo de conectividad con el servicio externo de IA, el estado cambia, provocando que la UI reaccione mostrando diálogos de alerta con opciones de reintento.

9. Análisis de flujo y cambios de estado

Se programaron interacciones nativas avanzadas:

- **Deslizar para Descartar:** Todas las vistas secundarias presentadas en modalidad de hoja modal, como el detalle de planes de estudio o los salarios, admiten el gesto vertical de arrastre hacia abajo para cerrarse suavemente y regresar a la jerarquía anterior sin necesidad de botones de retroceso explícitos.
- **Deslizamiento Horizontal:** En el panel de historial y listados de carreras guardadas, el usuario puede arrastrar horizontalmente una celda para revelar acciones rápidas nativas, permitiendo borrar un diagnóstico local o fijar una carrera como prioritaria en un solo movimiento.
- **Pulsación Larga Contextual:** Al mantener presionada una burbuja de mensaje dentro

del Chatbot Vocacional, se despliega un menú contextual flotante nativo con desenfoque de fondo, ofreciendo atajos para copiar la explicación de la IA o solicitar una profundización automática sobre ese punto en específico.

10. Dispositivos objetivo, tamaños y orientaciones

La aplicación está construida para su ejecución nativa y exclusiva dentro del ecosistema de dispositivos móviles de Apple.

- **Plataforma Base:** Nativos iOS 16.0 o superior. El uso estricto de SwiftUI asegura que la base de código sea escalable de forma coherente para su futura portabilidad nativa a iPadOS.
- **Manejo Responsivo:** Implementación basada en contenedores flexibles (GeometryReader, VStack, HStack) y el sistema de diseño nativo de restricciones. Se garantiza una legibilidad y distribución perfecta sin truncamiento de textos en pantallas compactas de 5.4 pulgadas hasta pantallas extendidas de 6.9 pulgadas.
- **Control de Orientación:**
 - Para asegurar la ergonomía y comodidad durante la escritura a una sola mano en el teclado virtual, las vistas operativas principales P01 a P07, incluyendo el Chatbot Vocacional, tienen su orientación estrictamente bloqueada en Modo Vertical (*Portrait*).
 - De manera exclusiva, el Dashboard de Resultados P08 y las retículas curriculares P09 permiten la rotación dinámica a Modo Horizontal para facilitar la visualización expandida de los polígonos de radar y tablas comparativas de materias.

11. Maqueta final y vistas de alta fidelidad

El sistema de diseño implementado para studAI adopta una estética moderna, limpia y minimalista, alineada con las guías de *Human Interface Guidelines* de Apple. Se priorizó un tema claro (Light Mode) con superficies blancas y un uso estratégico del espacio negativo para reducir la carga cognitiva del estudiante.

Paleta de Color Base y Sistema de Diseño:

- **Fondo General:** Gris claro mate, proporcionando un lienzo neutro que no cansa la vista.
- **Superficies y Tarjetas (Cards):** Blanco puro con sombras directas muy suaves para generar profundidad y jerarquía.
- **Acento Primario:** Azul Marino Profundo (utilizado en textos de títulos, iconos principales y botones de acción), transmitiendo formalidad y confianza institucional.

- **Acento Secundario:** Degradados dinámicos de Turquesa a Verde Esmeralda (utilizados en barras de progreso, iconos de la IA y porcentajes de afinidad), aportando un toque tecnológico y juvenil.
- **Navegación:** Implementación de un *Tab Bar* inferior flotante en forma de píldora translúcida, aislando las tres áreas principales ("IA & Enfoque", "Perfil", "Roadmap").

Descripción Visual de Pantallas Clave:

Vista 1: Panel de Perfil y Recomendaciones (P04) Presenta un diseño limpio sobre un fondo gris claro mate. En la parte superior, destaca una tarjeta blanca con esquinas redondeadas que contiene la información del usuario junto a un avatar circular. El cuerpo de la vista se divide en tres secciones modulares:

1. **Rutas de Carrera Recomendadas:** Una lista de tarjetas blancas que muestran la carrera sugerida (ej. "Ingeniería en IA") acompañadas de una etiqueta tipo píldora en color verde que indica el porcentaje exacto de compatibilidad (98%).
2. **Etiquetas del Orientador:** Chips interactivos que resaltan las habilidades blandas detectadas ("Pensamiento Lógico", "Resolución de Problemas").
3. **Afinidad por Áreas de Estudio:** Gráficos de barras horizontales rellenos con un degradado turquesa a verde esmeralda. En la parte inferior de la pantalla se ubica una barra de navegación flotante en forma de píldora translúcida para alternar entre áreas de la app.

Vista 2: Chatbot Vocacional en Vivo (P06) Muestra una interfaz conversacional sumamente limpia y minimalista. Las respuestas del mentor IA se alinean a la izquierda dentro de burbujas blancas con texto oscuro, acompañadas por un ícono circular turquesa con destellos que denota su origen algorítmico. Los mensajes del usuario se alinean a la derecha manteniendo la misma estética clara. El área de ingreso de texto está encapsulada en una píldora blanca redondeada justo por encima de la barra de navegación flotante, acompañada de un botón de envío circular en color azul marino profundo con una flecha hacia arriba.

Vista 3: Roadmap Vocacional (P08) Esta pantalla abandona la gráfica poligonal para presentar una estructura de línea de tiempo vertical mucho más accionable. En la parte superior, unos filtros que permiten al usuario alternar entre sus carreras compatibles ("IA 98%", "Datos 78%"), seguidos de una barra lineal que indica el "Avance de tu Ruta". El cuerpo de la vista está compuesto por nodos conectados por una línea vertical; cada nodo es una tarjeta blanca interactiva ("Bases Matemáticas", "Introducción a Python"). Estas tarjetas incluyen una breve descripción de la habilidad, etiquetas de enfoque (ej. "Lógica", "Código") y botones de acción en color azul marino profundo que enlazan directamente a recursos de aprendizaje externos (ej. "Ver Khan Academy" o "Empezar Curso Gratis").

12. Maqueta final y vistas de alta fidelidad

La selección tecnológica asegura un desarrollo ágil, un rendimiento óptimo en el cliente móvil y un procesamiento robusto y escalable en el servidor web.

Capa Arquitectónica	Lenguaje / Tecnología	Justificación Técnica de la Elección
Frontend Móvil	Swift 6 / SwiftUI	Framework nativo y declarativo oficial de Apple. Permite un desarrollo veloz, máxima optimización de recursos gráficos y animaciones fluidas integradas por defecto.
Backend Middleware	Python 3.11 / Flask	Microframework web extremadamente robusto y ligero. Facilita la manipulación veloz de estructuras JSON y provee integración directa de bajo nivel con las librerías oficiales de IA.
Base de Datos	MongoDB (Atlas Cloud)	Motor NoSQL orientado a documentos. Su flexibilidad estructural permite almacenar los planes de estudio y absorber cambios en los perfiles curriculares sin migraciones complejas.
Motor de IA	Google Gemini API	Se implementa el modelo gemini-1.5-flash debido a su inmensa ventana de contexto conversacional, baja latencia de respuesta y soporte nativo para retornar esquemas JSON estructurados.
Entorno y Despliegue	Xcode / Docker / Render	Contenedores Docker para encapsular el entorno del middleware en Python, asegurando paridad absoluta entre desarrollo local y los servidores de producción en la nube.

13. Arquitectura, metodologías y patrones de diseño

13.1 Patrón Arquitectónico Base: MVVM (Model-View-ViewModel)

El cliente móvil implementa el patrón **MVVM** aprovechando el sistema de reactividad nativo del lenguaje.

- **Modelo:** Estructuras de datos puras de Swift que representan el dominio de la aplicación. Implementan el protocolo Codable para permitir su decodificación directa y automatizada desde JSON.
- **Vista:** Archivos exclusivos de interfaz gráfica en SwiftUI. Son completamente pasivos y carecen de lógica algorítmica; su único rol es reflejar visualmente el estado actual del ViewModel.
- **Modelo de Vista:** Clases gestoras del estado marcadas para observación reactiva. Orquestan la lógica de validación, transforman los datos del modelo para su presentación y ejecutan las llamadas asíncronas a los servicios de red.

13.2 Patrones de Diseño Implementados

- **Patrón Singleton:** Aplicado en el gestor central de comunicaciones de red, garantizando que exista una única instancia global de sesión HTTP en toda la aplicación para optimizar la reutilización de conexiones.
- **Patrón Repositorio:** Actúa como una capa de abstracción opaca entre el ViewModel y las fuentes de datos. El repositorio decide internamente si resuelve una solicitud extrayendo datos de la base local en SwiftData o realizando una llamada HTTP hacia la API de Flask.
- **Patrón Inyección de Dependencias:** Los ViewModels reciben sus gestores de red y persistencia como protocolos inyectados en sus inicializadores, facilitando la creación de pruebas unitarias mediante servicios simulados .

14. Administración de proyectos

El equipo **Bit Beast** implementó una versión optimizada del marco de trabajo ágil **Scrum**.

- **Estructura de Sprints:** Se establecieron iteraciones, permitiendo realizar avances en documentación y funcionalidades continuas, evaluar el estado real del código y pivotar rápidamente ante imprevistos técnicos.
- **Herramientas Operativas:**
 - **Figma:** Empleado como entorno para la maquetación colaborativa del diseño visual

- **Estrategia de Control de Versiones (GitFlow Simplificado):**
- Repositorio alojado en **GitHub**.
- Rama main bloqueada para contener exclusivamente código estable, probado y funcional listo para compilación de producción.
- Rama develop como punto de integración continua del equipo.
- Ramas efímeras de características (feature/login-view, feature/gemini-service) con políticas de protección que exigen la revisión y aprobación de código (*Code Review*) mediante *Pull Requests* por parte de al menos otro integrante del equipo antes de realizar la fusión.

15. Uso de inteligencia artificial en el desarrollo

Mejoramos de manera significativa la velocidad de codificación y la robustez de la arquitectura integrando herramientas de IA directamente en el ciclo de vida del software:

1. **Asistencia en la Escritura de Código:** Empleo de IA para generación rápida de código repetitivo y redacción automatizada de los mapeos de decodificación JSON para los modelos Codable de Swift.
2. **Generación de Datos de Prueba:** Para someter la interfaz gráfica a pruebas de carga visual y validar el comportamiento del layout ante grandes volúmenes de información sin requerir conexión continua a la base de datos real, se emplearon LLMs para generar de forma masiva datos ficticios.
3. **Redacción y Estructuración de Documentación Técnica:** Se aprovechó la capacidad de procesamiento de lenguaje natural de los modelos generativos para apoyar en la síntesis, estructuración y refinamiento del presente reporte técnico. La IA facilitó la articulación clara de conceptos complejos de ingeniería de software (como el flujo de datos MVVM y las justificaciones de arquitectura), garantizando un formato exhaustivo, coherente y con el rigor académico requerido para la entrega final.

16. Frameworks, kits y herramientas nativas

El desarrollo del cliente móvil se apoya de forma estricta en las librerías nativas provistas por el SDK oficial del sistema operativo, minimizando dependencias externas para garantizar la longevidad del código y reducir el tamaño del binario instalable:

- **SwiftUI:** Framework base utilizado para la construcción declarativa de toda la capa de presentación visual y la orquestación nativa de animaciones.
- **SwiftData:** Framework de persistencia moderna y nativa de Apple (reemplazo directo de CoreData), empleado para modelar, consultar y almacenar de manera relacional el historial de resultados del alumno localmente utilizando objetos Swift puros.

- **Charts nativo:** Librería nativa de alto rendimiento introducida en iOS 16, empleada para renderizar directamente mediante aceleración de hardware las gráficas de compatibilidad y los polígonos vectoriales de radar en el panel de resultados, ofreciendo soporte automático para herramientas de accesibilidad nativas (*VoiceOver*).

17. Permisos y consideraciones para la app store Frameworks, kits y herramientas nativas

Para que **studAI** supere de manera exitosa el estricto proceso de certificación y sea publicada comercialmente en la **App Store** de Apple, la arquitectura de software se alinea meticulosamente con las normativas oficiales de revisión (*App Store Review Guidelines*):

1. **Política de Privacidad Obligatoria (Lineamiento 5.1.1):** Es un requerimiento ineludible disponer de una URL pública y permanente que aloje la Política de Privacidad de la plataforma. Este enlace debe declararse en *App Store Connect* y estar visible dentro de la pantalla de autenticación de la app. El documento debe especificar que las respuestas del usuario se procesan de forma disociada para fines exclusivos de inferencia vocacional y garantizar que no se efectúa rastreo publicitario de terceros.
2. **Seguridad de Red Obligatoria (*App Transport Security - ATS*):** El sistema operativo iOS bloquea por defecto cualquier intento de conexión de red que no esté encriptado. El servidor de producción en Flask debe tener configurado un certificado SSL válido para asegurar que todas las transferencias de datos viajen exclusivamente a través del protocolo seguro HTTPS.
3. **Inicio de Sesión Nativo Obligatorio (Lineamiento 4.8):** Dado que la aplicación ofrece facilidades de autenticación mediante proveedores de terceros (Google Auth), las directrices de revisión **exigen de forma estricta e ineludible** implementar como alternativa nativa el sistema **Inicio de Sesión con Apple (*Sign in with Apple*)**. Este botón debe presentarse en la pantalla de acceso con exactamente la misma prominencia visual e idéntico tamaño que las demás opciones; omitir este lineamiento resulta en el rechazo inmediato y automático de la aplicación durante el proceso de certificación.

18. Equipo de trabajo y roles

Para abarcar eficientemente todas las capas de la arquitectura de software y cumplir en tiempo y forma con los hitos de la asignatura, las cargas de trabajo profesionales del equipo se distribuyeron de la siguiente manera:

Integrante del Equipo	Rol Principal Asignado	Responsabilidades Técnicas Detalladas
Carreón Blas Andrea	Lead Frontend Developer & UI/UX Designer	Creación del sistema visual en Figma, diseño de wireframes, maquetación nativa de las 10 pantallas en SwiftUI, implementación reactiva del patrón MVVM y gestión del flujo de navegación interno de la app.
Hernández Cortés Armando Daniel	Backend Systems Architect & Prompt Engineer	Integración segura de la API de Google Gemini y optimización de los Prompts.

19. Estimaciones de tiempo de desarrollo y costos

19.1 Desglose de Esfuerzo Temporal

La planificación técnica del proyecto contempla el esfuerzo acumulado requerido para llevar a cabo la solución.

Fase de Ingeniería	Actividades Clave Comprendidas	Duración Estimada
1. Diseño y UX/UI	Definición de flujos de usuario, creación del sistema visual en Figma.	2 semanas
2. Capa Backend & API	Construcción de endpoints REST en Flask, conexión nativa con MongoDB e implementación y securización de Gemini API.	2 semanas

Fase de Ingeniería	Actividades Clave Comprendidas	Duración Estimada
3. Frontend Nativo iOS	Codificación declarativa de las 10 pantallas en SwiftUI, lógica reactiva de ViewModels e interfaces conversacionales.	3 semanas
4. Integración y Datos	Enlace asíncrono cliente-servidor y almacenamiento.	1 semana
5. Pruebas y Certificación	Pruebas de regresión, depuración de memoria, corrección de fallos.	1 semana
TOTAL GLOBAL		9 Semanas

19.2 Proyección Financiera Operativa

Para fundamentar la viabilidad comercial del producto de software, se desglosan los costos directos equivalentes de producción y los gastos recurrentes de operación proyectados para una base activa inicial de **10,000 estudiantes**:

- **Costo Directo de Desarrollo de Software (Inversión en Capital Humano):**
 - Tarifa profesional promedio ponderada para roles de ingeniería y diseño móvil en México: **\$350.00 MXN por hora**.
 - Costo Total Equivalente de la Construcción del MVP: 300 horas × \$350.00 MXN = **\$105,000.00 MXN**.
- **Gastos Recurrentes de Infraestructura y Licenciamiento Cloud (Anualizado):**
 - Membresía Oficial del *Apple Developer Program* (Obligatoria para despliegue en App Store): \$99 USD anuales ≈ **\$1,700.00 MXN / año**.
 - Alojamiento en la Nube del Middleware Backend (Instancia PaaS de Producción en Render): \$30 USD mensuales ≈ **\$6,120.00 MXN / año**.
 - Base de Datos NoSQL en la Nube (MongoDB Atlas - Instancia Dedicada M10 de Producción): \$60 USD mensuales ≈ **\$12,240.00 MXN / año**.
 - Facturación Proyectada por Consumo de la API de Google Gemini (Estimando un volumen de 60,000 interacciones de análisis anuales): ≈ **\$4,800.00 MXN / año**.
- **Presupuesto Operativo de Soporte y Mantenimiento Evolutivo:**
 - Provisión financiera del 15% sobre el costo de desarrollo para absorción de

parches de compatibilidad de iOS y actualizaciones de retículas universitarias: ≈
\$15,750.00 MXN / año.

Costo total proyectado para el primer año de operación: \$145,610.00 MXN.

20. Referencias bibliográficas

1. Apple Inc. (2026). *Human Interface Guidelines: Designing for iOS*. Apple Developer Documentation. Recuperado de <https://developer.apple.com/design/human-interface-guidelines>
2. Apple Inc. (2026). *App Store Review Guidelines*. Apple Developer Support. Recuperado de <https://developer.apple.com/app-store/review/guidelines/>
3. Google LLC. (2026). *Gemini API: Prompting Guide, REST Reference and Structured Outputs*. Google AI for Developers Documentation. Recuperado de <https://ai.google.dev/docs>
4. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2da ed.). O'Reilly Media.
5. Instituto Mexicano para la Competitividad (IMCO). (2025). *Compara Carreras: Tendencias del mercado laboral, rentabilidad educativa y deserción universitaria en México*. IMCO Informes Sectoriales.
6. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
7. Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería del software: un enfoque práctico* (9na ed.). McGraw-Hill Interamericana.
8. Universidad Nacional Autónoma de México. (2025). *Oferta Académica y Planes de Estudio Vigentes*. Dirección General de Administración Escolar (DGAE UNAM).
9. Universidad Panamericana. (s.f.). *7 de cada 10 estudiantes no saben qué carrera elegir al salir del bachillerato*. Blog UP Prepaup. Recuperado de <https://blog.up.edu.mx/prepaup/femenil/7-de-cada-10-estudiantes-no-saben-que-carrera-elegir>
10. Universidad del Valle de México. (2025). *Señales de que deberías cambiar de carrera universitaria y el impacto de la desorientación*. Blog UVM. Recuperado de <https://blog.uvm.mx/senales-que-deberias-cambiar-de-carrera>

21. Anexo

A continuación se muestran 3 vistas de las pantallas de la aplicación

